

DeFuzz: Agentic Discovery of Silently-Failing Compiler Defenses via Cross-Mechanism Security Invariants

Anonymous Submission
Paper #XXX — Under Review

Abstract—Compiler-inserted defenses—stack canaries, `_FORTIFY_SOURCE`, CET/IBT, CFI, BTI, PAC, and shadow stacks—are the last line that must hold once a memory-safety bug is triggered. Their effectiveness depends not only on the correctness of the defense code but also on assumptions about the underlying instruction set architecture (ISA). When such an assumption breaks on a particular backend, the defense can fail silently: the program compiles without warning and runs with correct functionality, yet its security contract is no longer enforced. CVE-2023-4039, in which the stack protector of GCC is defeated on AArch64 in the presence of variable-length arrays, is one such case, and its cross-architecture siblings have persisted upstream for years.

The detection of silent failures is challenging due to two coupled reasons. The search space is a two-dimensional matrix of defense mechanism $\times$ ISA, each cell backed by an independent middle-end pass or backend template. Furthermore, failure adjudication is difficult: the binary neither crashes nor disagrees with other compilers; thus, crash-based and differential-oracle fuzzers register nothing. We call this the oracle gap.

We present DeFuzz, an agentic system that closes the oracle gap and searches the matrix directly. First, we systematize the safety invariants that each defense must satisfy—distilled from compiler source comments, ABI specifications, and confirmed historical bugs—into a machine-checkable oracle; every checker returns a four-state verdict and stays sound by grounding each bug claim in a deterministic binary or runtime signal rather than in model output. Second, we introduce a cross-mechanism invariant-generation pipeline that treats a confirmed root cause in one mechanism as a probe, retrieves structurally analogous code in another mechanism, and instantiates a new invariant behind two static grounding gates; from 25 confirmed defense bugs and 24 probes over GCC 16.1 it yields 11 new cross-mechanism invariants, none duplicating an existing seed. Third, we build an explicitly-orchestrated agentic loop: a deterministic pipeline drives build, coverage, and oracle in a fixed order and invokes agents only at fixed positions, where they communicate through a versioned blackboard rather than free-form dialogue. Unlike free-running agent systems, this design yields reproducible traces, per-edge ablation, and bug claims that trace back to deterministic evidence; a checker-bound ISA routing scheme collapses the mechanism $\times$ ISA Cartesian product into a single semantic decision. On GCC and LLVM, DeFuzz discovered [TBD: N] silent-failure defects, of which [TBD: M] were confirmed upstream.

I. Introduction

The number of publicly disclosed vulnerabilities has grown for a decade; the CVE database now registers more than twenty thousand new entries per year and

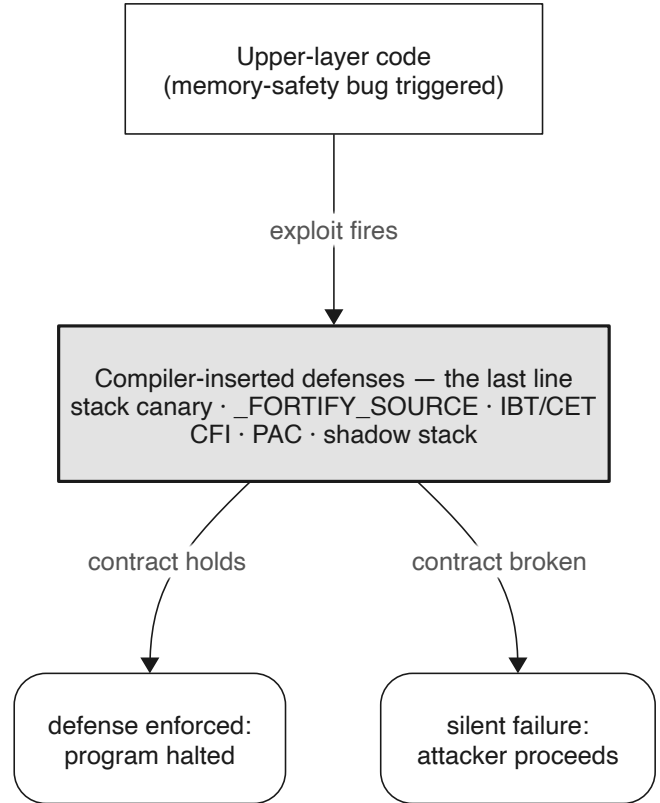


Fig. 1. Compiler-inserted defenses as the runtime last line: they must hold once an upper-layer memory-safety bug is triggered.

the rate is still rising. Major vendors have conceded that manual auditing and secure-coding disciplines alone are insufficient to eliminate memory-safety bugs in upper-layer code. As a consequence, the behavior of a program following a bug trigger has become increasingly critical: stack canaries, `_FORTIFY_SOURCE`, Indirect Branch Tracking (IBT), Control-Flow Integrity (CFI), Pointer Authentication (PAC), and shadow stacks are deployed jointly by the compiler and the runtime as a backstop intended to hold even when an exploit fires.

Compiler-emitted defenses share a common working model: at compile time the compiler inserts check instructions, layout constraints, or metadata so that any run-time

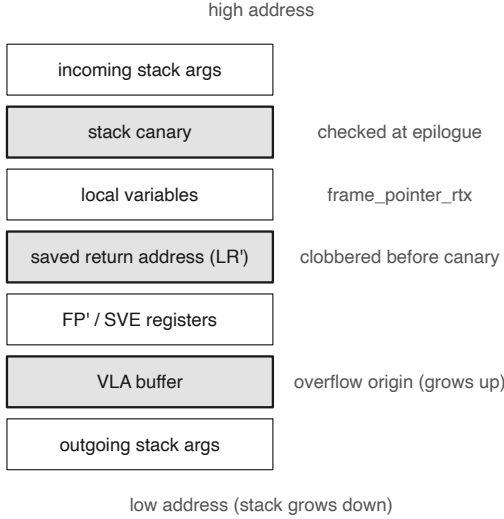


Fig. 2. CVE-2023-4039 buggy stack frame: canary above the VLA, return address below it, so an overflow bypasses the check.

deviation from the intended behavior is caught and the program is terminated. The stack canary is the canonical example—a random sentinel is placed between the buffer and the saved return address, and is compared against its original value before the function returns.

However, defense mechanisms themselves consist of code, which is susceptible to bugs. Furthermore, the effectiveness of a defense depends not only on the correctness of its implementation but also on the underlying ISA. As the number of ISAs keeps growing, the cross-architecture attack surface widens, and the rigor of defense implementations within compilers becomes correspondingly harder to guarantee.

A. Silent failure

CVE-2023-4039 is a representative case. The `-fstack-protector` mechanism of GCC was defeated on AArch64 for a long time: when a function uses a variable-length array (VLA) or `alloca`, the compiler places the canary above the VLA while the saved return address sits below it, so an overflow rewrites the return address before the canary is ever checked. The bug had existed since early GCC versions and was only analyzed and fixed by outside researchers in 2023. Figure 2 shows the buggy frame layout.

This class of failure has a double-silent character: it raises no error at compile time (toolchain and functional tests see nothing) and produces correct functional results at run time (program behavior sees nothing), yet the security contract is already broken. We call this a silent failure: the compiled artifact appears to carry a complete line of defense, but an attacker can bypass it entirely.

	x86-64	AArch64	RISC-V	LoongArch	MIPS	...
Stack Canary						...
<code>_FORTIFY_SOURCE</code>						...
IBT / SHSTK						...
...	...	...	...	...	...	...
	Unexplored	Historical CVE	Reported (pending)	Confirmed & merged		

Fig. 3. The defense mechanism $\times$ ISA matrix. Each cell is an independent backend/pass; silent failures hide in individual cells.

B. Two coupled challenges

Systematically exposing silent failures presents two intertwined challenges.

Large search space. As shown in Figure 3, the compiler must uphold a security contract across a two-dimensional space of many defense mechanisms (canary, `_FORTIFY_SOURCE`, IBT, CFI, ...) and many ISAs (x86-64, AArch64, RISC-V, LoongArch, ...). Each cell maps to an independent middle-end pass or backend template, and any detail can decide whether the mechanism takes effect. Homologous omissions are not rare: PR-96191, a sibling of CVE-2023-4039 fixed in 2020, covered only some architectures and left several fallback backends untouched, so the same failure persisted silently on other targets for years. Our preliminary experiments already confirmed several such homologous failures with the GCC upstream.

Difficulty in failure adjudication: the oracle gap. Even when a test reaches the relevant code path, “is the defense contract still satisfied” is not directly observable: the program does not crash, its output agrees with other compilers, and everything looks normal while the contract has already been violated. We call the static or dynamic properties a defense must satisfy at run time its safety invariants (e.g., the canary must lie between any overflow-reachable stack object and the return address); breaking them is a silent failure. Existing automated methods are oblivious to this phenomenon: crash- or differential-output tools such as Csmith and YARPGen trigger no anomaly, and coverage-guided fuzzing lacks the oracle to judge the security status of each cell.

C. Our approach

DeFuzz combines three stages. First, it generates safety invariants through two complementary paths: a segmented chain-of-thought (CoT) review extracts candidates from defense-related corpora, while a RAG pass uses confirmed historical bugs as probes to find implementations with homologous root causes across mechanisms and compiler backends and generalize them into additional invariants. Second, accepted invariants are translated into static or dynamic checkers. Third, the orchestrator assigns one

invariant-checker pair at a time, expands the pair to the ISAs declared by the checker, and runs an oracle-grounded agent loop. The model proposes seeds and feedback; deterministic binary or runtime evidence decides whether a bug exists.

D. Contributions

- An oracle for silent failure. A sound, machine-checkable oracle that fills the silent-failure oracle gap and grounds confirmed bug reports in reproducible evidence (§VI).
- Systematized security invariants. A bottom-up taxonomy over 468 catalogued invariants across more than two dozen defenses, each in a machine-decidable static/dynamic form. Segmented CoT review provides broad corpus coverage, while historical-bug RAG yields 11 new invariants (§IV, §V).
- An explicitly-orchestrated agent system. A deterministic pipeline with agents at fixed positions linked through a versioned blackboard, yielding reproducible, ablatable, and auditable runs—in contrast to free-running agent systems (§VII).
- An agentic loop. Oracle-grounded, coverage-guided seed generation scheduled by invariant-checker pairs, with ISA applicability derived from checker metadata (§VII).

II. Background and Motivation

A. Compiler defenses and the mechanism $\times$ ISA matrix

A compiler-emitted defense inserts, at compile time, either extra check instructions (canary comparison, CFI target checks), layout constraints (guard placement, shadow-stack slots), or metadata (BTI landing pads, PAC signing) so that a run-time deviation is detected and the program is halted. Whether the inserted defense is actually effective is decided on a per-backend basis: stack layout, register allocation, and instruction encoding all differ across ISAs, and each is realized by a distinct pass or target template. The matrix in Figure 3 therefore has genuinely independent cells, and a defense that is correct in one cell can be silently broken in the next.

B. Silent failure: a real case

Returning to CVE-2023-4039: the effectiveness of the stack protector depends on the invariant “the canary lies between every overflow-reachable stack object and the saved return address.” On AArch64 with a VLA, that invariant is violated by construction, yet nothing in the build or in a functional test reveals it. PR-96191 demonstrates a recurrence of this pattern: a fix that lands on some targets but not on fallback backends leaves the invariant violated elsewhere. Both cases exemplify the same phenomenon—a complete-looking defense that an attacker bypasses.

C. Why existing methods miss it

Static analysis is constrained by patterns: silent failures exhibit heterogeneous forms, and most confirmed cases involve idiosyncratic violations, rendering pattern matching difficult to generalize. Crash and differential fuzzing monitor surface-level signals—whether the program crashes or disagrees with another compiler—which by construction cannot catch a silent logic failure; this is the oracle gap. Free-running agent fuzzers can find bugs but sacrifice reproducibility: with an agent deciding the control flow, when to compile, and when to declare a bug, trajectories cannot be replayed, ablations cannot be run, and a bug claim cannot be traced back to a deterministic cause.

D. Scope and threat model

We target the compiler’s own defense implementation on GCC and LLVM. A finding is a silent failure: a compiled artifact whose defense contract is violated while it compiles cleanly and runs correctly. Following our project constraint, we describe only the violated invariant (the falsification surface) and do not construct working exploits. The source-comment and RAG corpus is pinned to GCC 16.1 to keep evidence and reproduction anchored to one tree.

III. System Overview

DeFuzz separates offline knowledge construction from online bug search. The offline stage reads compiler source, specifications, and historical bug records. A segmented CoT review provides broad candidate extraction, while historical-bug RAG targets implementations that may share a known root cause. Candidates that survive evidence grounding and deduplication enter the invariant catalog and are implemented as static or dynamic checkers. Confirmed findings can be added to the historical-bug pool as new retrieval probes.

The online stage tests the compiler rather than the generated program. At run initialization, the orchestrator enumerates the checker catalog into a fixed queue. Each target denotes one invariant-checker pair; checker metadata supplies the applicable ISAs. The target remains fixed until its round or wall-clock budget is exhausted. Within each round, the Generator produces a C seed and a hard-wired pipeline executes routing $\rightarrow$ build $\rightarrow$ coverage $\rightarrow$ oracle. Pass, NotApplicable, and Error enter the Feedback branch before the next round; Error remains visible as an infrastructure diagnostic and consumes the current round without becoming a bug. A Fail invokes the Minimizer and is rechecked against the same checker. All intermediate states and results are versioned for replay and audit.

Figure 5 shows one target-specific iteration end to end; the following sections detail each stage.

IV. Security Invariants for Compiler Defenses

A. What is a security invariant

A security invariant is a property a defense must satisfy for it to be effective; violating it means the mechanism

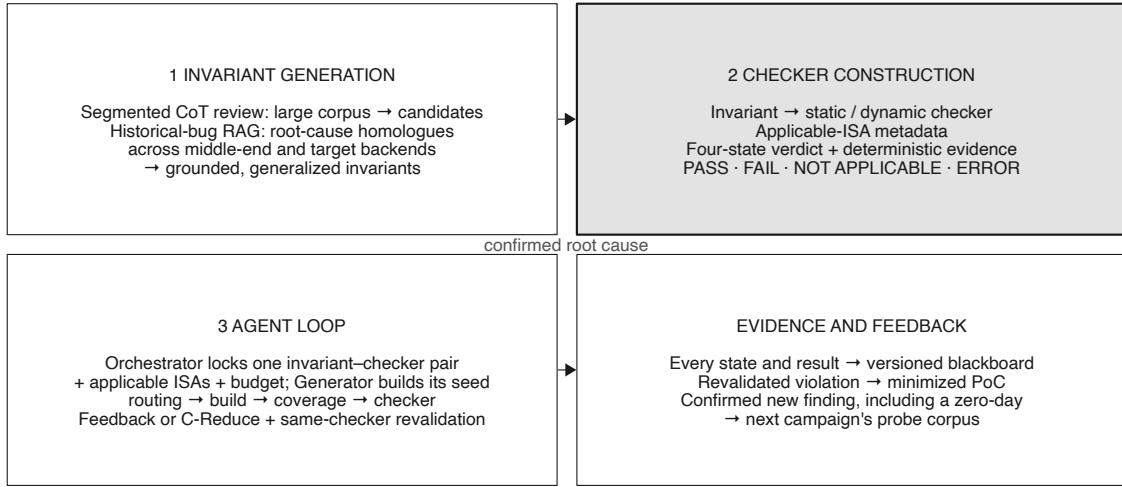


Fig. 4. DeFuzz technical route: corpus-grounded invariant generation feeds programmatic checkers, which define the targets of an explicitly orchestrated agent loop.

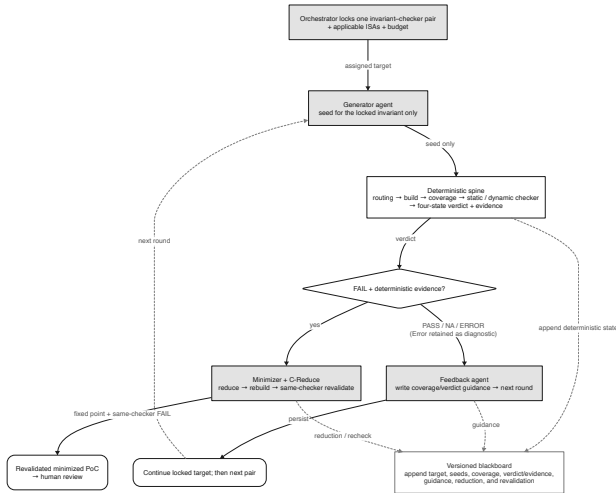


Fig. 5. Agent loop for one invariant-checker target and its applicable ISAs: generate → build → coverage → check → feedback or minimization.

has been silently weakened in the compiled artifact or at run time, whether or not the code itself contains an overt bug. Each invariant is recorded with a machine-verifiable statement and, separately, an observation field that names only the externally observable phenomenon of a violation—not how to detect it. The separation of the phenomenon from its detection recipe is deliberate: “the `__stack_chk_fail` symbol is missing from the binary” is a phenomenon, whereas “look it up in the ELF `.dynsym`” is a detection mechanism that belongs to the oracle implementation.

B. Sources and segmented coarse extraction

The invariant archive draws on three source classes: (i) compiler source comments and assertions, (ii) compiler and ABI documentation, and (iii) confirmed historical bugs and their patches. Every record carries a uniform schema—ID, statement, compiler, version, target, source_kind, evidence, version_sensitivity, and observation—so that a downstream consumer (oracle, static analysis, CI) can decide how to use each invariant independently.

The corpus is too large for a single model context. Related evidence also lies in distant source files, specifications, and patches. We therefore partition the material by defense mechanism, compiler phase, and function boundary. A segmented CoT prompt [1] asks the model to identify the protected asset, activation conditions, required security property, and externally observable violation in each segment. This pass produces candidates rather than accepted invariants. Every candidate must retain its source evidence and pass the grounding checks in §V.

C. A bottom-up taxonomy

Rather than imposing a top-down category scheme, we cluster the 468 catalogued invariants data-first to avoid preset bias, and read the root-cause families off the clusters (Figure 6). Three families recur across mechanisms and ISAs: (a) exit-time sensitive-register/state residue, in which a fast or exceptional exit bypasses the register/state clearing that a defense relies on; (b) stack-clash frame-size truncation, in which a forced integer narrowing flips a very large frame size negative and the probing loop is skipped; and (c) RISC-V large-address materialization, in which an unintended sign extension while splitting or synthesizing a large address/offset shifts the computed address.

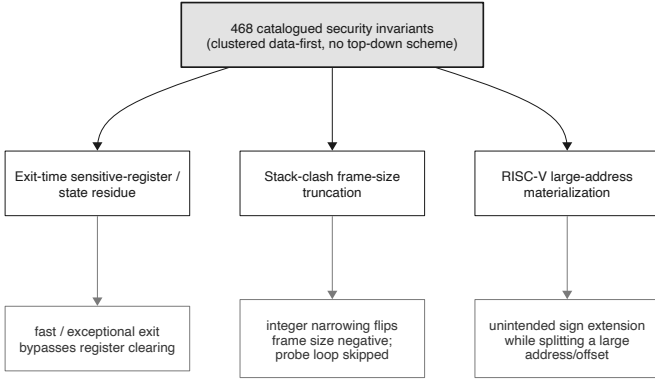


Fig. 6. Bottom-up root-cause families over 468 invariants; three families recur across mechanism and ISA.

D. Machine-checkable form: static vs dynamic

Each invariant is classified by whether its violation can be decided from the un-executed binary (symbols, sections, disassembly, strings)—static—or only by running it and observing exit status, output flags, or register/memory snapshots—dynamic. The choice is guided by four dimensions: observability, decisiveness, cost, and static/dynamic attribution. When an invariant has both a statically observable part and a more precise dynamic part, we split it into two checkers rather than forcing one classification.

V. Corpus-Grounded Invariant Generation

A. Why direct review needs retrieval

Segmented review gives the LLM broad corpus coverage, but it does not reliably cluster fragments that encode the same root cause. The relevant evidence may be separated by mechanism names, compiler phases, target backends, or years of patch history. RAG [2] therefore provides a complementary high-yield path: a confirmed historical bug becomes an explicit probe for related implementations rather than an association the model must discover while reading unrelated segments. RAG raises the chance of recovering bugs that share a known root cause; it does not guarantee completeness. Segmented review covers the remaining corpus for invariants with no historical probe.

B. Historical-bug RAG and root-cause generalization

The RAG stage first distills a confirmed bug into a mechanism-agnostic failure signature. Retrieval then searches the indexed middle-end and target-backend corpus for structurally or semantically related implementations. We avoid entity-only retrieval based on API or symbol names because such queries tend to remain within the original mechanism. Instead, the query represents the abstract operation, violated assumption, and protected asset. This formulation allows one bug to expose analogous sites across mechanism and backend boundaries.

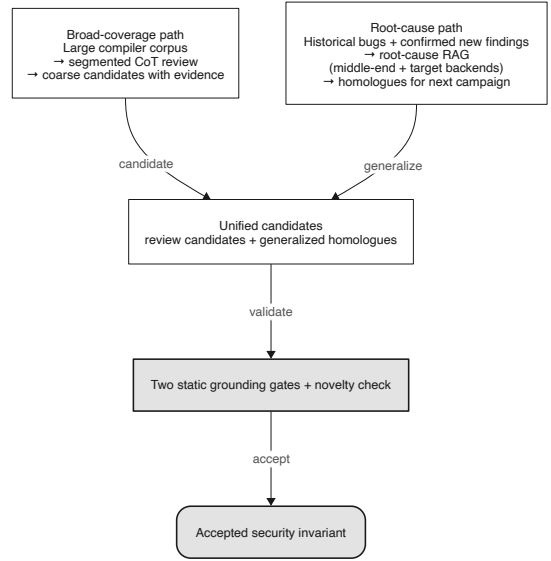


Fig. 7. Corpus-grounded invariant generation: segmented CoT review and historical-bug RAG feed a shared grounding and novelty filter; confirmed new findings return as retrieval probes.

C. Pipeline: distill → analogy → specialize → entailment

A confirmed root cause is first distilled into a mechanism-agnostic description. The analogy step decides whether a retrieved code block instantiates the same failure shape. The specialize step instantiates the invariant on the concrete target source, and entailment checks that the statement follows from the cited evidence. The analogy gate rejects lexical matches that do not share the root-cause structure before specialization. Figure 7 shows how segmented review and the RAG path feed a shared grounding and novelty filter.

D. Retrieval backends and current yield

Within this single RAG stage, we evaluate sparse BM25 and dense doubao-embedding-vision retrieval over the same 24 probe seeds and the same 4,496 GCC-16.1 source chunks. BM25 excels on distinctive identifiers (`ix86_zero_call_used_regs`, `SUBREG_PROMOTED`, `CONST_HIGH_PART`); the dense retriever pulls in semantically isomorphic blocks whose lexical anchors are weak (the CMSE clear-set constructor, the RISC-V saturating truncation). Deduplicating by (seed_id, chunk_id) and statement, the two paths yield 5 in the intersection, 4 BM25-only, and 2 embedding-only, for 11 in the union (Figure 8); all 11 are marked `is_novel` after a novelty check (threshold 85.0). Table I summarizes.

E. Grounding gates and reproducibility

Candidates from both segmented CoT review and historical-bug RAG enter the same acceptance path. An invariant is accepted only after two static grounding gates confirm the cited evidence supports its statement and

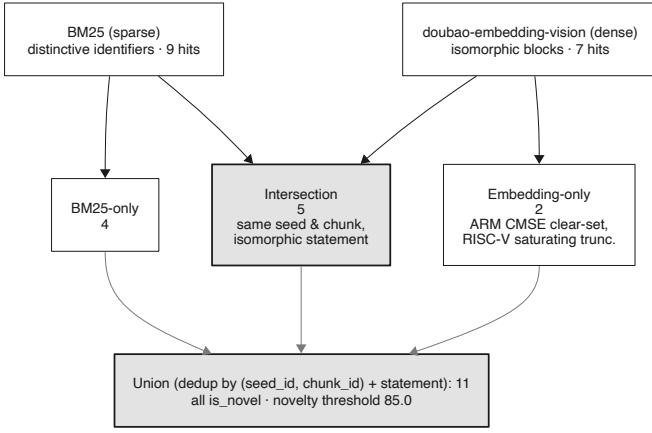


Fig. 8. Union/dedup of BM25 and embedding retrieval: 5 intersection, 4 BM25-only, 2 embedding-only, 11 union.

TABLE I
Cross-mechanism invariants by retrieval backend (24 probes, 4,496 GCC-16.1 chunks).

Category	Count	Note
Intersection (BM25 $\cap$ embed)	5	same seed & chunk, isomorphic statement
BM25-only	4	embed missed the block for that seed
Embedding-only	2	BM25 missed; dense retrieval added
Union (deduplicated)	11	9 + 7 - 5; all is_novel

observation, and after the novelty check (threshold 85.0) confirms it does not duplicate an existing seed or manual rule. Because the dense retriever returns slightly different vectors per call, we cache query vectors keyed by query text so that top- k ordering—and hence the accepted set—reproduces across runs. After a new finding, including a zero-day, is reproduced and its root cause is confirmed, we serialize it in the same schema and add it to the probe pool. The next campaign can then search for further instances of that root cause.

VI. Oracle: From Invariants to Verdicts

A. From an invariant to a checker

An accepted invariant is not yet an oracle. We first derive a checker contract that fixes the preconditions, observable signal, applicable ISAs, and evidence required for a verdict. If the violation is decidable from an unexecuted binary, the invariant becomes a static checker over symbols, sections, or disassembly. If adjudication requires exit status, output, or register and memory state, it becomes a dynamic checker. When both views are useful, we keep two checkers rather than weakening a deterministic claim into a heuristic.

Each checker returns a four-state verdict (pass / fail / not-applicable / error). NotApplicable is returned when

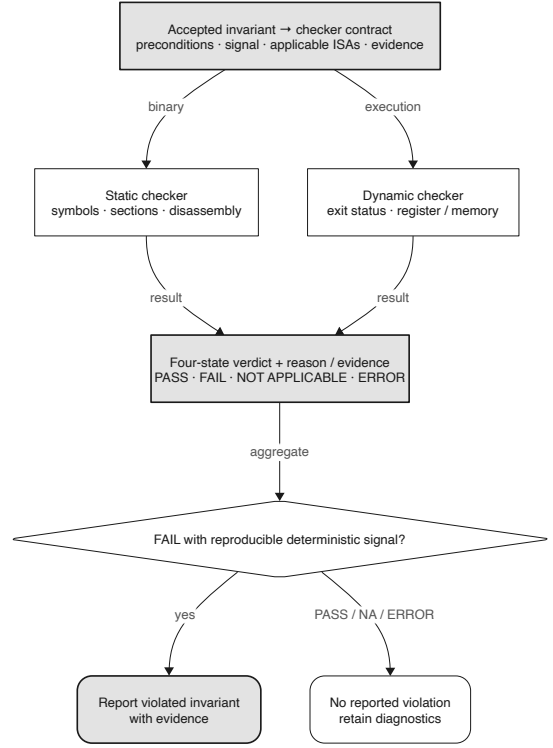


Fig. 9. An accepted invariant is translated into a static or dynamic checker; four-state verdicts pass through a deterministic-evidence gate before any finding is confirmed.

the mechanism is off or the seed does not exercise it, and is ignored by the aggregation layer. A finding is counted as confirmed only when a deterministic checker fails or an execution differential reproduces, never on the strength of model output alone (Figure 9).

When a programmatic checker cannot decide an invariant-ISA cell and returns NotApplicable or Error, an optional LLM fallback can inspect extracted disassembly and symbol evidence. A high-confidence model Fail may promote the run’s provisional aggregate to violated and send the seed to minimization. The result remains explicitly marked as LLM-derived and is excluded from confirmed findings until a deterministic checker or execution differential reproduces it.

B. Mechanism aggregator and flag profiles

A per-mechanism aggregator combines checker verdicts: any Fail raises a mechanism-level violation carrying its evidence. On/off flag profiles let the oracle adjudicate differentially (defense enabled vs. disabled), and because the checkers key on binary and runtime signals rather than on the producing compiler, GCC and LLVM share the same checkers.

C. Checker metadata as a single source of truth

Each checker declares three metadata fields: which ISAs it binds, whether it is single-ISA or differential, and

whether it is cheap or expensive. This metadata is the single source of truth read by both the gRPC and the MCP adapter, and it is what makes the ISA routing in §VII a table lookup rather than an agent decision.

VII. Agentic Loop

A. Design principles

The orchestrator, rather than an agent, owns target selection and control flow. At run initialization it constructs a fixed queue of invariant-checker pairs. For each target, checker metadata determines the applicable ISAs, and the target remains fixed for a configured round or time budget. Within a round, the pipeline follows the same order: generate → routing → build → coverage → oracle → route. Agents are invoked only at fixed positions and communicate through shared state. The Generator proposes; the oracle stack adjudicates, and confirmed findings require deterministic evidence.

B. Explicit orchestration and the blackboard

The agents form a closed loop: feedback guides the next Generator round, the oracle adjudicates, a non-violation returns to feedback, and a violation goes to the Minimizer. All communication flows through a versioned blackboard rather than direct agent-to-agent messages (Figure 10). The blackboard holds the current invariant-checker target, its applicable ISAs and budget, the seed corpus and lineage, cumulative coverage, oracle verdicts, feedback guidance, and reduction results. This design provides three properties: reproducibility (pin a blackboard version and any agent’s input is fully determined), ablatability (each linkage edge can be switched off independently), and auditability (a bug claim traces back through the blackboard to a deterministic cause).

C. Three agents

The Generator (invoked at the start) carries read-only source-search and invariant-query tools and outputs a seed for the assigned invariant-checker pair; coverage is not an agent-reportable tool but is measured by the coverage node and fed back, so the agent cannot fabricate coverage. The feedback agent (invoked on a non-violation) is a context-isolated subagent that turns the coverage delta, checker identity, and the Pass/NotApplicable verdict into next-round guidance written back to the blackboard. The minimizer (invoked on a violation) shrinks the PoC with deterministic delta debugging (C-Reduce) as the workhorse and the LLM only for semantic guidance, so that reduction does not accidentally delete the structure that triggers the bug. Each candidate reduction is compiled and rechecked with the same checker before it is retained.

D. Invariant-checker scheduling and ISA binding

Each target in the deterministic queue denotes one invariant and its checker. The routing node expands that

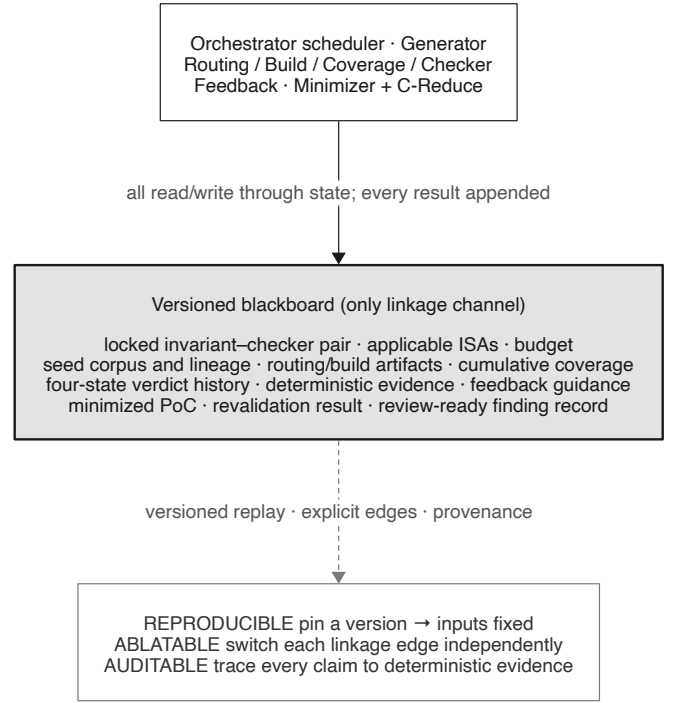


Fig. 10. The blackboard: all inter-agent linkage flows through versioned shared state, enabling reproduce / ablate / audit.

checker to every ISA in its metadata; the Generator never chooses either the target invariant or an ISA (Figure 11). A differential checker always expands to its full ISA set so that a cross-ISA signal cannot be pruned by model behavior. Cheap static checkers remain enabled as a side path, but they do not change the Generator’s assigned target. When a checker returns Pass or NotApplicable, the Feedback agent records the relevant context and the same pair continues until its budget expires. Error follows the same non-bug path, remains visible as an infrastructure diagnostic, and consumes the current round. A Fail transfers the seed and deterministic evidence to the Minimizer. The blackboard records both branches before the orchestrator advances.

VIII. Implementation

The Go core (cmd/defuzz-core) exposes `grpc_server.go` (build, coverage, oracle, and checker-metadata services) and `mcp_server.go` (the read-only tools `search_source`, `query_invariants`, `coverage_diff`, `creduce_run`, `compile_exec`); the oracle, checkers, and metadata SSOT live under `internal/oracle`. The Python orchestrator (`defuzz_loop`) holds `graph.py` (fixed edge order + conditional routing + enumeration cursor), `state.py` (the blackboard), `audit.py` (per-run directory + checkpoint), `routing.py` (the checker → ISA matrix expansion), the three agents, and the deterministic node wrappers. Each execution generates a dedicated per-run directory with `checkpoints.sqlite` (the versioned state chain) and mani-

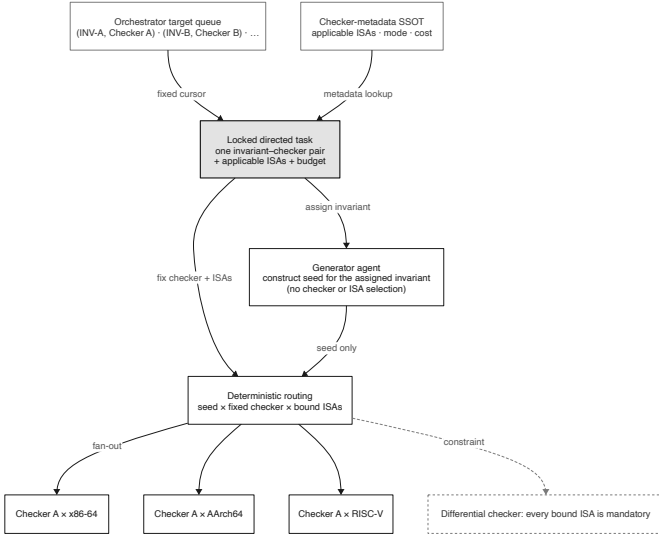


Fig. 11. Deterministic target scheduling: one invariant-checker pair remains fixed for its budget, while checker metadata expands the applicable ISAs and cheap static checkers remain enabled.



Fig. 12. Ablation: contribution of oracle grounding, coverage feedback, each inter-agent edge, and checker routing to hit rate/cost.

fest.json (git SHA, toolchains, checker catalog, LLM config, ablation flags), which the inspect / replay / trace-bug subcommands consume for reproduction. The target queue stores checker IDs, each of which names the corresponding invariant-checker pair; the cursor advances only after that target’s round or time budget is exhausted.

IX. Evaluation

Setup. Targets are an instrumented GCC 16.1, a cross-GCC, and LLVM; ISAs are x86-64, AArch64, RISC-V, and LoongArch, with cross-architecture execution under QEMU user mode. We answer five research questions.

a) RQ1 — Real bugs: DeFuzz found [TBD: N] silent-failure defects on GCC/LLVM; [TBD: M] were confirmed upstream ([TBD: CVE IDs]).

b) RQ2 — Invariant generation quality: The historical-bug RAG path produced 11 union invariants (5 intersection, 4 BM25-only, 2 embedding-only), all is_{novel}; the analogy gate filtered roughly 94% of BM25 hits as lexical collisions (Table I).

c) RQ3 — Ablation: We toggle coverage feedback, oracle grounding, and each inter-agent edge, and compare checker routing against a full Cartesian fan-out (Figure 12). [TBD: ablation numbers]

d) RQ4 — Explicit orchestration: Pinning a black-board version reproduces a run’s trajectory exactly; we compare stability and hit rate against free orchestration. [TBD: reproducibility numbers]

e) RQ5 — Agentic vs. fuzz loop: We measure the gain on “reaching silent-failure bugs” over the prior coverage-driven fuzz loop. [TBD: comparison numbers]

X. Discussion and Limitations

DeFuzz describes only violated invariants and does not build exploits; the RAG corpus is pinned to GCC 16.1; dense retrieval is non-deterministic and requires query-vector caching to reproduce; and checker routing is a performance optimization guarded by always-on static checkers, not a correctness mechanism.

XI. Related Work

Compiler bug finding. Csmith [3], YARPGen [4], and EMI [5] find miscompilations with crash or differential oracles, which by construction miss silent defense failures. Silent/security bugs in compilers. Studies such as “Silent Bugs Matter” [6] taxonomize compiler-introduced security bugs but do not build an agentic detector. LLM/RAG property generation. PropertyGPT [7] generates and verifies smart-contract properties via retrieval-augmented generation; SpecAuditor [8] detects specification violations by semantic generalization—we deliberately reject its entity-driven retrieval in favor of abstract-failure-mode transfer. LLM-agent fuzzing. AgentFuzz [9] applies directed grey-box fuzzing to LLM agents; in contrast, our agents are explicitly orchestrated rather than free-running.

XII. Conclusion

An invariant-grounded oracle turns “agents can find bugs” from a one-off demo into a reproducible method for silent defense failures across the mechanism × ISA matrix. By pairing a sound oracle with an explicitly-orchestrated agentic loop and checker-bound ISA routing, DeFuzz makes such bugs both findable and auditable.

References

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [3] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.
- [4] V. Livinskii, D. Babokin, and J. Regehr, “Random testing for C and C++ compilers with YARPGen,” in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.
- [5] V. Le, M. Afshari, and Z. Su, “Compiler validation via equivalence modulo inputs,” in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

- [6] G. A. Di Luna et al., “Silent bugs matter: A study of compiler-introduced security bugs,” in Proceedings of the USENIX Security Symposium, 2023.
- [7] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, “PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation,” in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.
- [8] M. Lin and H. Chen, “SpecAuditor: Detecting specification violations via semantic generalization,” in IEEE Symposium on Security and Privacy (S&P), 2026.
- [9] F. Liu et al., “Testing the robustness of LLM-based agents via directed greybox fuzzing,” in Proceedings of the USENIX Security Symposium, 2025.